

- Boolean Algebra
 - When we learned numbers like 1, 2, 3, we also then learned how to add, multiply, etc. with them. Boolean Algebra is operations that we can do with 0's and 1's. Computers do these operations ALL THE TIME and they are basic building blocks of computation inside your computer program.
- Axioms, laws, theorems
 - We need to know some rules about how those 0's and 1's can be operated together. There are similar axioms to decimal number algebra, and there are some laws and theorems that are good for you to use to simplify your operation.

HOW DOES BOOLEAN ALGEBRA FIT INTO THE BIG PICTURE?

- It is part of the Combinational Logic topics (memoryless)
 - Different from the Sequential logic topics (can store information)
- Learning Axioms and theorems of Boolean algebra
 - Allows you to do design logic functions
 - Allows you to know how to combine different logic gates
 - Allows you to simplify or optimize on the complex operations

- A Boolean algebra comprises...
 - A set of elements B
 - Binary operators $\{+, \bullet\}$ Boolean sum and product
 - A unary operation $\{ '\}$ (or $\{ \}$) example: A' or A —
- ...and the following axioms
 - 1. The set B contains at least two elements $\{a, b\}$ with $a \neq b$
 - 2. Closure: $a+b$ is in B $a \bullet b$ is in B
 - 3. Commutative: $a+b = b+a$ $a \bullet b = b \bullet a$
 - 4. Associative: $a+(b+c) = (a+b)+c$ $a \bullet (b \bullet c) = (a \bullet b) \bullet c$
 - 5. Identity: $a+0 = a$ $a \bullet 1 = a$
 - 6. Distributive: $a+(b \bullet c) = (a+b) \bullet (a+c)$ $a \bullet (b+c) = (a \bullet b) + (a \bullet c)$
 - 7. Complementarity: $a+a' = 1$ $a \bullet a' = 0$

DIGITAL (BINARY) LOGIC IS A BOOLEAN ALGEBRA

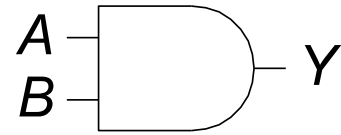
- Substitute
 - $\{0, 1\}$ for B
 - AND for $\bullet$ Boolean Product
 - OR for $+$ Boolean Sum
 - NOT for $'$
- All the axioms hold for binary logic
- Definitions
 - Boolean function
 - Maps inputs from the set $\{0,1\}$ to the set $\{0,1\}$
 - Boolean expression
 - An algebraic statement of Boolean variables and operators

DIGITAL (BINARY) LOGIC IS A BOOLEAN ALGEBRA

- **Perform logic functions:**
 - inversion (NOT), AND, OR, NAND, NOR, etc.
- **Single-input:**
 - NOT gate, buffer
- **Two-input:**
 - AND, OR, XOR, NAND, NOR, XNOR
- **Multiple-input**

2 INPUT LOGIC GATES

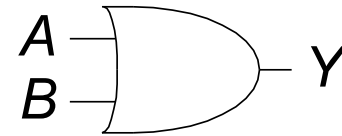
AND



$$Y = AB$$

A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

OR

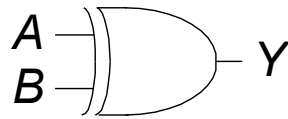


$$Y = A + B$$

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1

2 INPUT LOGIC GATES

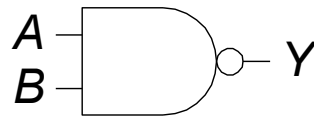
XOR



$$Y = A \oplus B$$

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

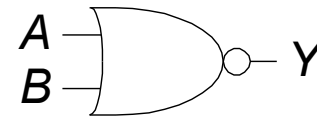
NAND



$$Y = \overline{AB}$$

A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0

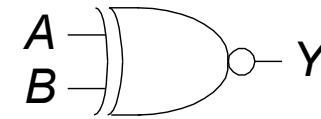
NOR



$$Y = \overline{A + B}$$

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	0

XNOR

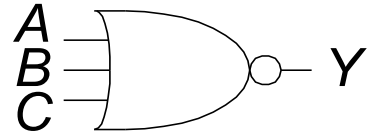


$$Y = \overline{A \oplus B}$$

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	1

3 INPUT LOGIC GATES

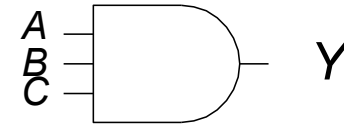
NOR3



$$Y = \overline{A+B+C}$$

A	B	C	Y
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	0

AND3



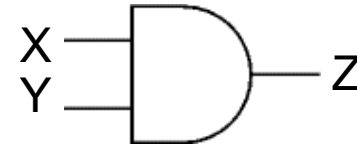
$$Y = ABC$$

A	B	C	Y
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	1

- Multi-input XOR: Odd parity

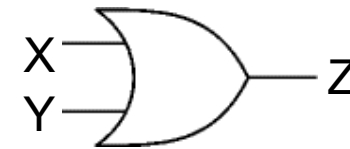
LOGIC GATES AND TRUTH TABLES

• AND $X \bullet Y$ XY



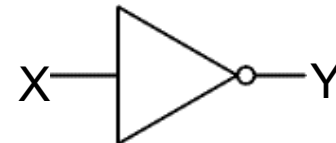
X	Y	Z
0	0	0
0	1	0
1	0	0
1	1	1

• OR $X + Y$



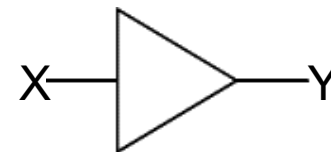
X	Y	Z
0	0	0
0	1	1
1	0	1
1	1	1

• NOT X X' —



X	Y
0	1
1	0

• Buffer X



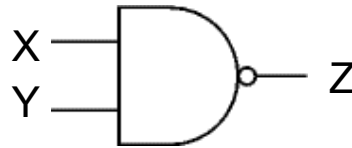
X	Y
0	0
1	1

LOGIC GATES AND TRUTH TABLES (CON'T)

- NAND

$$\overline{X \bullet Y}$$

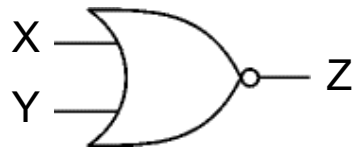
$$\overline{XY}$$



X	Y	Z
0	0	1
0	1	1
1	0	1
1	1	0

- NOR

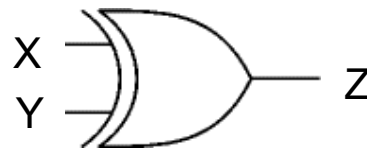
$$\overline{X + Y}$$



X	Y	Z
0	0	1
0	1	0
1	0	0
1	1	0

- XOR

$$X \oplus Y$$



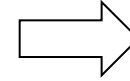
X	Y	Z
0	0	0
0	1	1
1	0	1
1	1	0

- XNOR

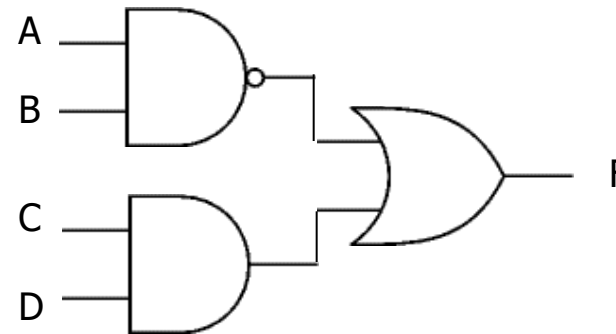
$$\overline{X \oplus Y}$$



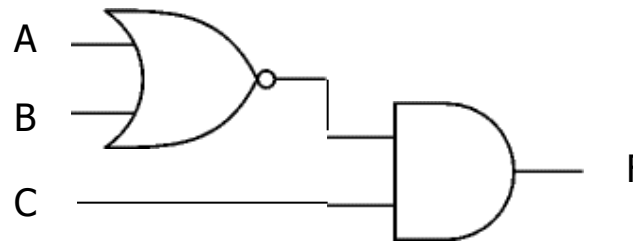
X	Y	Z
0	0	1
0	1	0
1	0	0
1	1	1



- Example: $F = (A \bullet B)' + C \bullet D$



- Example: $F = C \bullet (A + B)'$



LOGIC FUNCTIONS AND BOOLEAN ALGEBRA

- Any logic function that is expressible as a truth table can be written in Boolean algebra using $+$, $\bullet$, and $'$

X	Y	Z
0	0	0
0	1	0
1	0	0
1	1	1

 $Z = X \bullet Y$

X	Y	X'	Z
0	0	1	0
0	1	1	1
1	0	0	0
1	1	0	0

 $Z = X' \bullet Y$

X	Y	X'	Y'	$X \bullet Y$	$X' \bullet Y'$	Z
0	0	1	1	0	1	1
0	1	1	0	0	0	0
1	0	0	1	0	0	0
1	1	0	0	1	0	1

 $Z = (X \bullet Y) + (X' \bullet Y')$

TWO KEY CONCEPTS

- Duality (a meta-theorem— *a theorem about theorems*)
 - All Boolean expressions have logical duals
 - Any theorem that can be proved is also proved for its dual
 - Replace: $\bullet$ with $+$, $+$ with $\bullet$, 0 with 1, and 1 with 0
 - Leave the variables unchanged
- de Morgan's Theorem
 - Procedure for complementing Boolean functions
 - Replace: $\bullet$ with $+$, $+$ with $\bullet$, 0 with 1, and 1 with 0
 - Replace all variables with their complements

USEFUL LAWS AND THEOREMS

Identity:	$X + 0 = X$	Dual: $X \bullet 1 = X$
Null:	$X + 1 = 1$	Dual: $X \bullet 0 = 0$
Idempotent:	$X + X = X$	Dual: $X \bullet X = X$
Involution:	$(X')' = X$	
Complementarity:	$X + X' = 1$	Dual: $X \bullet X' = 0$
Commutative:	$X + Y = Y + X$	Dual: $X \bullet Y = Y \bullet X$
Associative:	$(X+Y)+Z=X+(Y+Z)$	Dual: $(X\bullet Y)\bullet Z=X\bullet(Y\bullet Z)$
Distributive:	$X\bullet(Y+Z)=(X\bullet Y)+(X\bullet Z)$	Dual: $X+(Y\bullet Z)=(X+Y)\bullet(X+Z)$
Uniting:	$X\bullet Y+X\bullet Y'=X$	Dual: $(X+Y)\bullet(X+Y')=X$

USEFUL LAWS AND THEOREMS (CON'T)

Absorption:	$X + X \bullet Y = X$	Dual: $X \bullet (X + Y) = X$
Absorption (#2):	$(X + Y') \bullet Y = X \bullet Y$	Dual: $(X \bullet Y') + Y = X + Y$
de Morgan's:	$(X + Y + \dots)' = X' \bullet Y' \bullet \dots$	Dual: $(X \bullet Y \bullet \dots)' = X' + Y' + \dots$
Duality:	$(X + Y + \dots)^D = X \bullet Y \bullet \dots$	Dual: $(X \bullet Y \bullet \dots)^D = X + Y + \dots$

Multiplying & factoring: $(X + Y) \bullet (X' + Z) = X \bullet Z + X' \bullet Y$
Dual: $X \bullet Y + X' \bullet Z = (X + Z) \bullet (X' + Y)$

Consensus: $(X \bullet Y) + (Y \bullet Z) + (X' \bullet Z) = X \bullet Y + X' \bullet Z$
Dual: $(X + Y) \bullet (Y + Z) \bullet (X' + Z) = (X + Y) \bullet (X' + Z)$

DE MORGAN'S THEOREM

- Use de Morgan's Theorem to find complements
- Example: $F = (A+B) \cdot (A'+C)$, so $F' = (A' \cdot B') + (A \cdot C')$

A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

A	B	C	F'
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0

- Logic Gates and Truth Table
 - Now you know 0's and 1's and the basic Boolean algebra, now you are ready to go back and forth between truth table, Boolean expression, and logic gates. This ability to go back and forth is an extremely useful skill designing and optimizing computer hardware.
- Implementing Logic Functions
 - Now with these basic tools you learned, you can “implement” logic functions. You learned algebra in junior/high school and you use it for many things you do now. We use Boolean algebra to implement logic functions that are used in the computers. And these logic functions are used by computer programs you write.

WHAT IS THE OPTIMAL GATE REALIZATION?

- We use the axioms and theorems of Boolean algebra to “optimize” our designs
- Design goals vary
 - Reduce the number of gates?
 - Reduce the number of gate inputs?
 - Reduce number of chips and/or wire?
- How do we explore the tradeoffs?
 - Logic minimization: Reduce number of gates and complexity
 - Logic optimization: Maximize speed and/or minimize power
 - CAD tools

MINIMAL SET

- We can implement any logic function from NOT, NOR, and NAND
 - Example: $(X \text{ and } Y) = \text{not } (X \text{ nand } Y)$
- In fact, we can do it with only NOR or only NAND
 - NOT is just NAND or NOR with two identical inputs

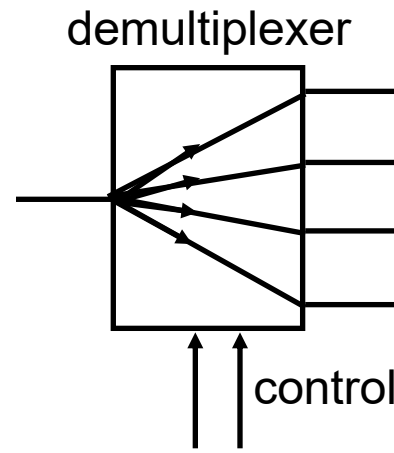
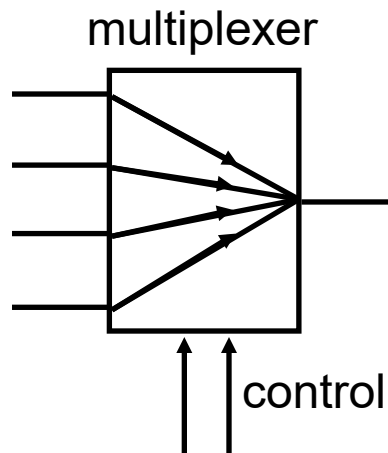
X	Y	X nor Y
0	0	1
1	1	0

X	Y	X nand Y
0	0	1
1	1	0

- NAND and NOR are duals: Can implement one from the other
 - $X \text{ nand } Y = \text{not } ((\text{not } X) \text{ nor } (\text{not } Y))$
 - $X \text{ nor } Y = \text{not } ((\text{not } X) \text{ nand } (\text{not } Y))$

SWITCHING-NETWORK LOGIC BLOCKS

- Multiplexer (MUX)
 - Routes one of many inputs to a single output
 - Also called a *selector*
- Demultiplexer (DEMUX)
 - Routes a single input to one of many outputs
 - Also called a *decoder*



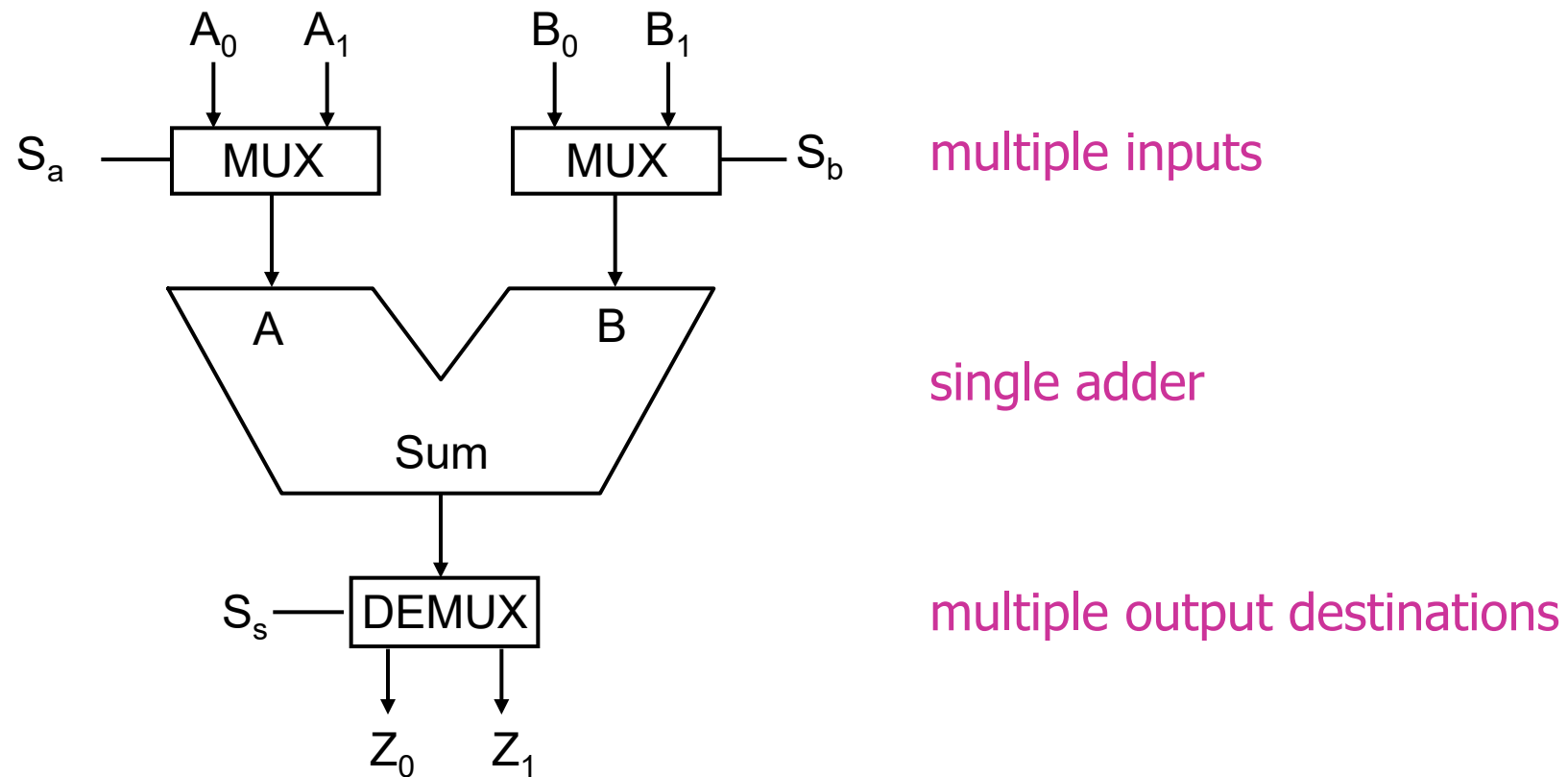
We construct these devices from:

- logic gates
- networks of transistor switches

- Multiplexers/Demultiplexers
 - If you had the ability to select which input to operate, the same part of a circuit can be used multiple times. So if you have a lot of inputs and all of them are supposed to go through same complex logic functions, you can save a lot of space on your circuit board by using a multiplexer.
 - Then you will also need a demultiplexer to decode the output coming out in serial into separate output ports.

“WHY”: SHARING COMPLEX LOGIC FUNCTIONS

- Share an adder: Select inputs; route sum

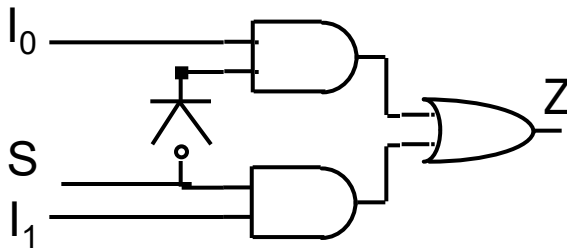


MULTIPLEXERS

- Basic concept
 - 2^n data inputs; n control inputs ("selects"); 1 output
 - Connects one of 2^n inputs to the output
 - "Selects" decide which input connects to output
 - Two alternative truth-tables: **Functional** and **Logical**

Example: A 2:1 Mux

$$Z = SIn_1 + S'In_0$$



Functional truth table

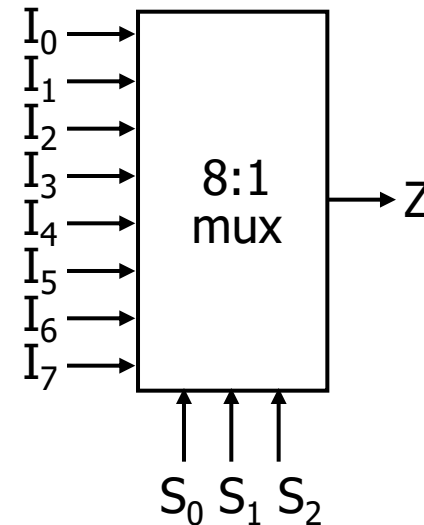
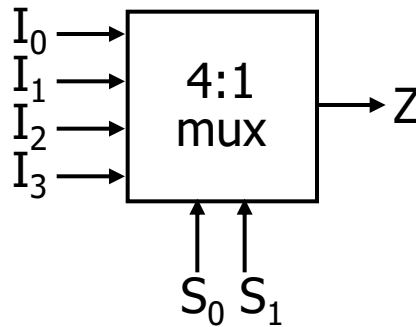
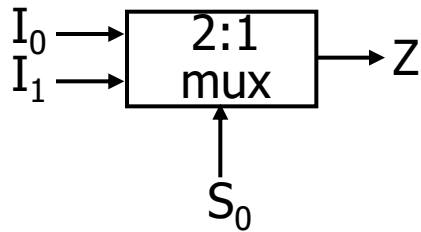
S	Z
0	In_0
1	In_1

Logical truth table

In_1	In_0	S	Z
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

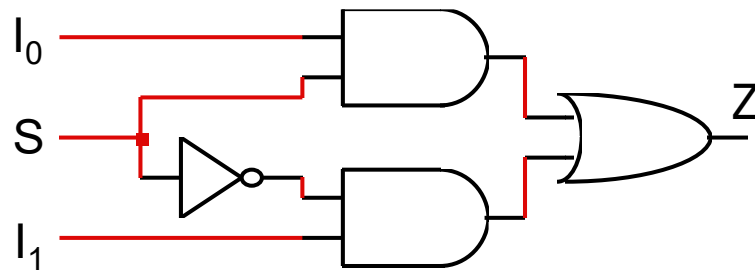
MULTIPLEXERS (CON'T)

- 2:1 mux: $Z = S'_0 I_{n_0} + S_0 I_{n_1}$
- 4:1 mux: $Z = S'_0 S'_1 I_{n_0} + S'_0 S_1 I_{n_1} + S_0 S'_1 I_{n_2} + S_0 S_1 I_{n_3}$
- 8:1 mux: $Z = S'_0 S'_1 S'_2 I_{n_0} + S'_0 S'_1 S_2 I_{n_1} + \dots$

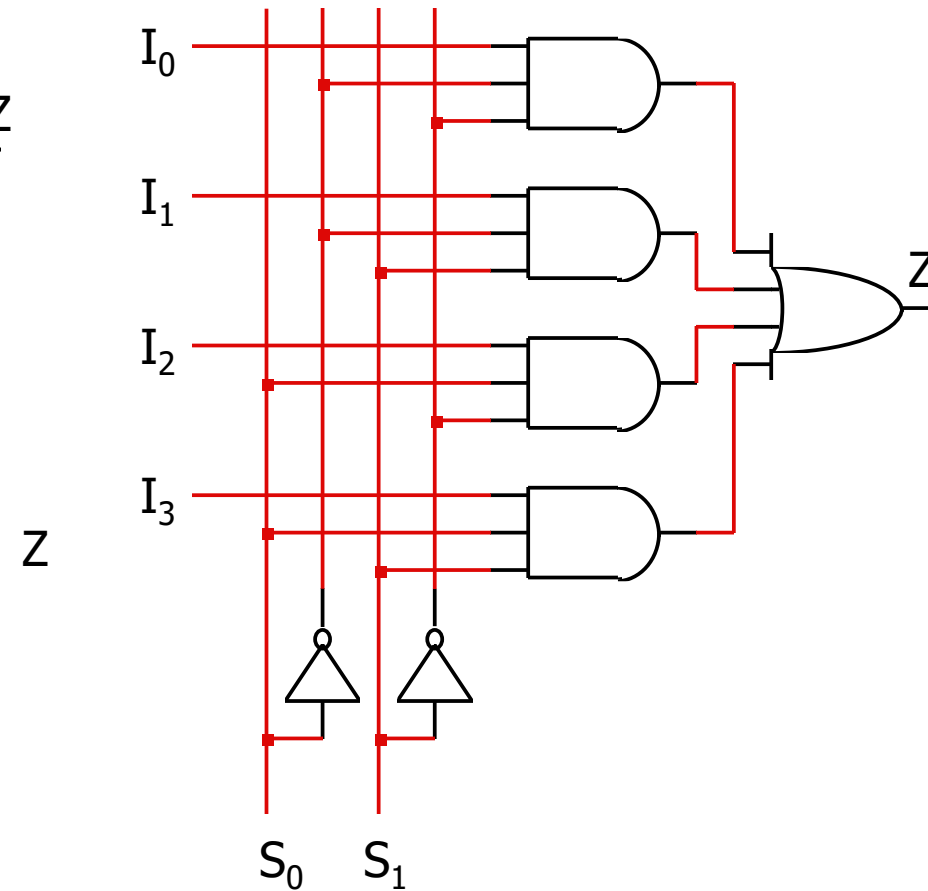


LOGIC-GATE IMPLEMENTATION OF MULTIPLEXERS

2:1 mux



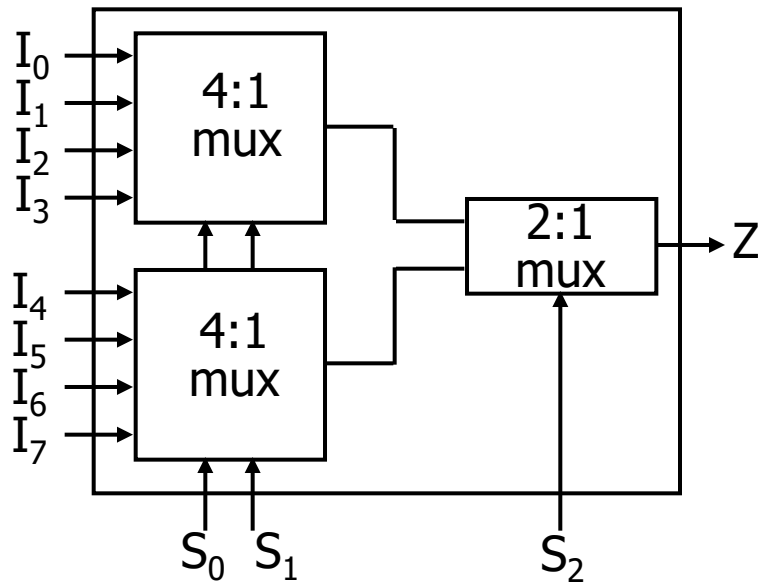
4:1 mux



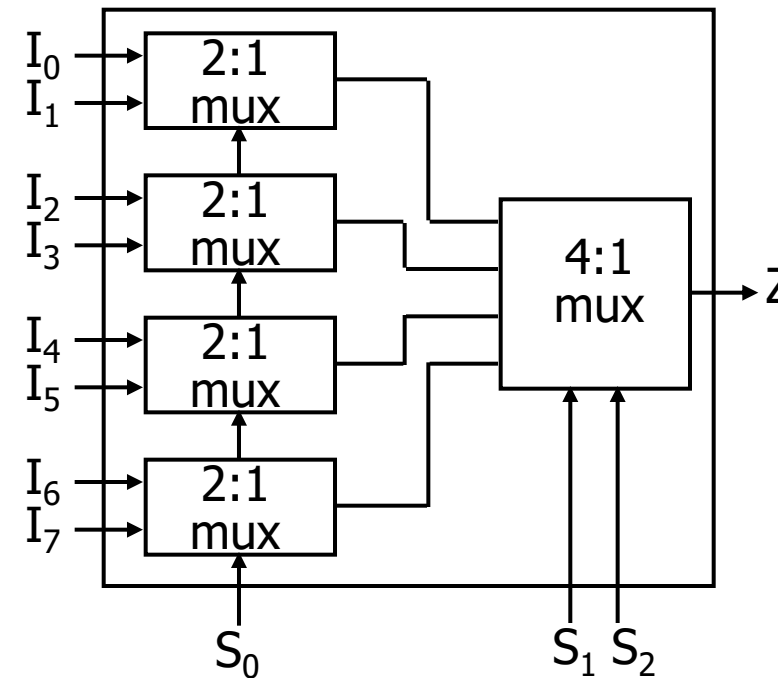
CASCADING MULTIPLEXERS

- Can form large multiplexers from smaller ones
 - Many implementation options

8:1 mux



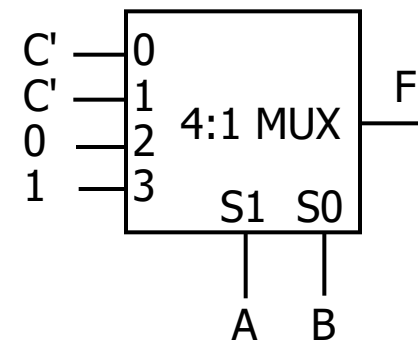
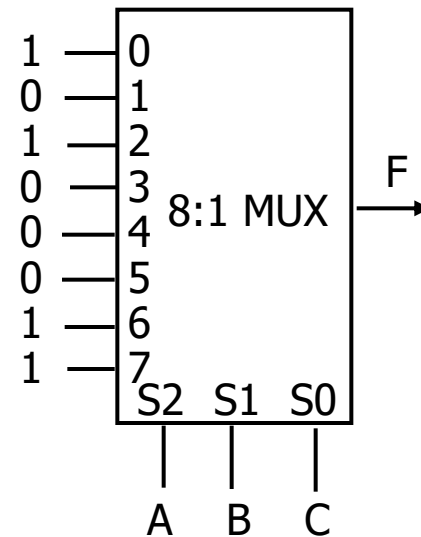
8:1 mux



MULTIPLEXERS AS GENERAL-PURPOSE LOGIC

- A $2^n:1$ mux can implement any function of n variables
 - A lookup table
 - A $2^{n-1}:1$ mux also can implement any function of n variables
- Example: $F(A,B,C) = m_0 + m_2 + m_6 + m_7$
 $A'B'C' + A'BC' + ABC' + ABC$
 $AB(0) + AB(1)$

A	B	C	F
0	0	0	1
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1



MULTIPLEXERS AS GENERAL-PURPOSE LOGIC

- Implementing a $2^{n-1}:1$ mux as a function of n variables
 - $(n-1)$ mux control variables $S_0 - S_{n-1}$
 - One data variable S_n
 - Four possible values for each data input: $0, 1, S_n, S_n'$
 - Example: $F(A,B,C,D)$ implemented using an 8:1 mux

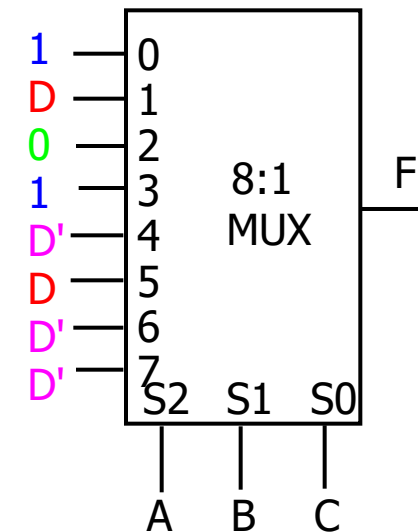
AB		A			
		00	01	11	10
CD	00	1	0	1	1
	01	1	0	0	0
	11	1	1	0	1
	10	0	1	1	0

B

D

Choose A,B,C as
control variables

Choose D as a
data variable



DEMULTIPLEXERS (DEMUX)

- Basic concept
 - Single data input; n control inputs (“selects”); 2^n outputs
 - Single input connects to one of 2^n outputs
 - “Selects” decide which output is connected to the input
 - When used as a decoder, the input is called an “enable” (G)

1:2 Decoder:

$$\text{Out0} = G \bullet S'$$

$$\text{Out1} = G \bullet S$$

2:4 Decoder:

$$\text{Out0} = G \bullet S1' \bullet S0'$$

$$\text{Out1} = G \bullet S1' \bullet S0$$

$$\text{Out2} = G \bullet S1 \bullet S0'$$

$$\text{Out3} = G \bullet S1 \bullet S0$$

3:8 Decoder:

$$\text{Out0} = G \bullet S2' \bullet S1' \bullet S0'$$

$$\text{Out1} = G \bullet S2' \bullet S1' \bullet S0$$

$$\text{Out2} = G \bullet S2' \bullet S1 \bullet S0'$$

$$\text{Out3} = G \bullet S2' \bullet S1 \bullet S0$$

$$\text{Out4} = G \bullet S2 \bullet S1' \bullet S0'$$

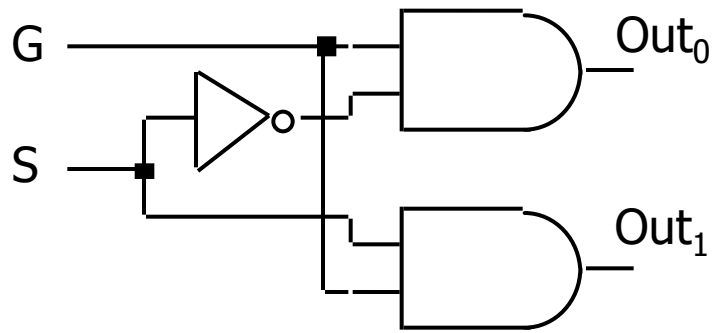
$$\text{Out5} = G \bullet S2 \bullet S1' \bullet S0$$

$$\text{Out6} = G \bullet S2 \bullet S1 \bullet S0'$$

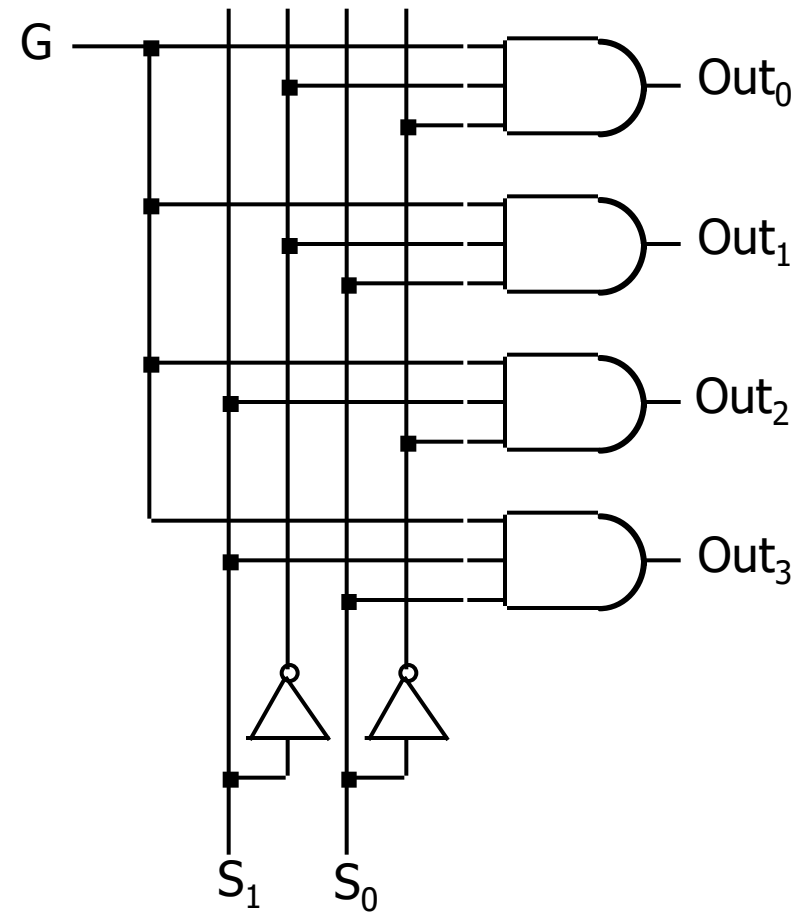
$$\text{Out7} = G \bullet S2 \bullet S1 \bullet S0$$

LOGIC-GATE IMPLEMENTATION OF DEMULTIPLEXERS

1:2 demux

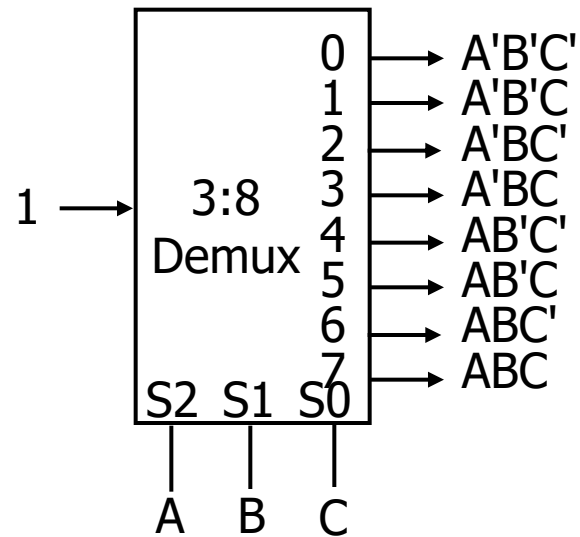


2:4 demux



DEMULTIPLEXERS AS GENERAL-PURPOSE LOGIC

- A $n:2^n$ demux can implement any function of n variables
 - DEMUX as logic building block
 - Use variables as select inputs
 - Tie enable input to logic 1
 - Sum the appropriate minterms (extra OR gate)



demultiplexer “decodes”
appropriate minterms
from the control signals

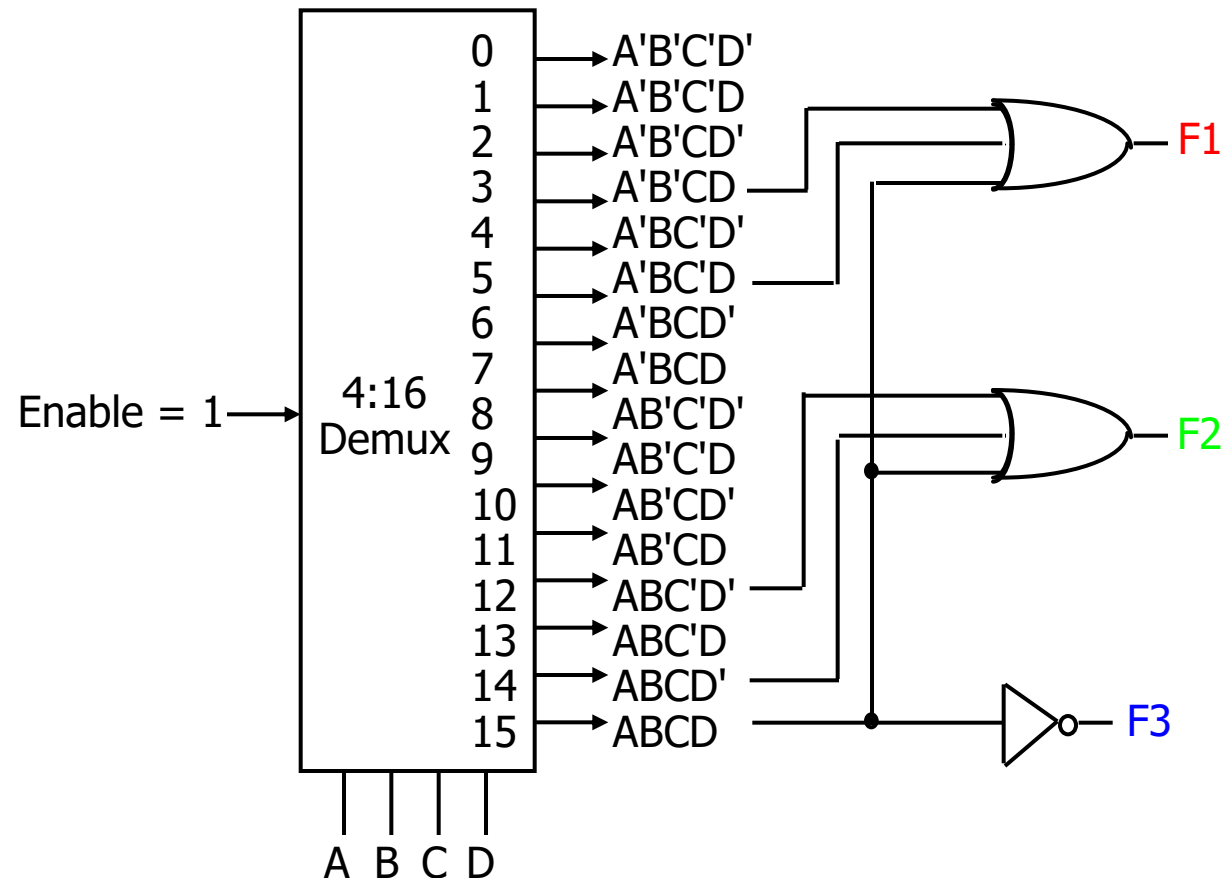
DEMULTIPLEXERS AS GENERAL-PURPOSE LOGIC

Example

$$F1 = A'BC'D + A'B'CD + ABCD$$

$$F2 = ABC'D' + ABC$$

$$F3 = (A'+B'+C'+D')$$



CASCADING DEMULTIPLEXERS

- 5:32 demux

